

Competitive Coding

Experiment $\rightarrow$ 7.1

1. Cheapest distance from source to destination with at most k stops:-

Procedure

- create adj list for each node and push destination and cost.
- create a queue and push source and 0 as its initial cost in it.
- Traverse for queue front while $k \geq 0$, and take its source and update minimum distance from source to destination.
- If distance of final destination node is INT_MAX , then return -1, else return $dist[destination]$.

```
vector <vector <pair <int, int>>> adj(n);
for (auto [u, v, cost] : vec) {
    adj[u].push_back({v, cost});
}
```

```
vector <int> mincost (n, INT_MAX);
queue <pair <int, int>> q;
q.push({src, 0});
while (!q.empty() && k >= 0) {
    auto [s, p] : adj[s];
    for (auto [v, p] : adj[s]) {
        if (1 + p < mincost[v]) {
            mincost[v] = 1 + p;
        }
    }
    q.push({v, mincost[v]});
}
```

}

k--;

}

return (mincost[dest] == INF_MAX)? -1 : mincost[dest];

Time complexity $\rightarrow (K \times E)$

Space complexity $\rightarrow (V + E)$